

GPU-Accelerated Cube Attack in Prime Fields

Extending the Kite Framework to target Arithmetization-Oriented Ciphers

Gaetano Bruno

May 8, 2026

Roma Tre University

Target Primitive: small-pSquare

The Context: Prime-Field Masking & SCA

- Boolean masking fails to provide security in **low-noise regimes**.
- *small-pSquare* is a TBC that operates natively in $\mathbb{F}_p$ to enable prime-field masking.
- The non-linear function $x \mapsto x^2$ supports efficient gadgets.
- Total algebraic degree grows exponentially ($\geq 4^r/8$). *chỉ là r?*

Our Attack Vector

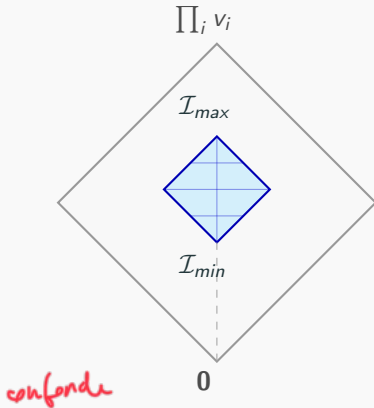
- **Goal:** Search for “weak” monomials in the ANF and exploit them to isolate affine relations in the key elements.
- **Method:** Massively parallel GPU Cube Attack.

The Method: Topological Search via the “Kite” Framework

To overcome the $\mathcal{O}(p^d)$ barrier, the Kite-Attack framework¹ proposes a **massively parallel, deterministic topological search**.

- $\mathcal{I}_{min}$: floor of the search space,
 $|\mathcal{I}_{min}| = \beta$.
- $\mathcal{I}_{max}$: ceiling of the search space,
 $|\mathcal{I}_{max}| = \alpha$.
- $\mathcal{I}_{free} := \mathcal{I}_{max} \setminus \mathcal{I}_{min}$ free variables set.

$$\mathcal{N}_{\text{config}} = 3^{\alpha-\beta}$$



~~Hasse~~ diagram of the Boolean sub-lattice

¹M. Bernaschi et al., *Kite attack: reshaping the cube attack for a flexible GPU-based maxterm search*, Journal of Cryptographic Engineering, 2019.

Kernel 1: Foundational Tensor Generation

To avoid evaluating the cipher $3^{\alpha-\beta}$ times from scratch, we precompute base cube sums over $\mathcal{I}_{min}$. Hardware mapping:

$$1 \text{ Boolean Mask in } \mathcal{I}_{free} \iff 1 \text{ GPU Warp}$$

Kernel 2: Cube Reconstruction & Affine Testing

Cubes of dimension $d \in \{\beta, \dots, \alpha\}$ are reconstructed from the foundational tensor. An in-register affine test is executed on the superpoly $q(\underline{k})$:

$$e = q(\underline{k}_a) + q(\underline{k}_b) + q(\underline{k}_c) - 2q(\underline{k}_0) - q(\underline{k}_a + \underline{k}_b + \underline{k}_c) \pmod{p}$$

Kernel 3: Formal Superpoly Extraction

Once a maxterm is successfully validated, the pipeline compute the superpoly coefficients and outputs the linear equation.

Cryptanalytic Results

The topological search successfully penetrated up to **Round 4** and cleanly isolated valid linear superpolys. This enabled the exact algebraic recovery of **3/16** secret key elements.

β	α	Maxterm	Superpoly
6	22	$x_0 x_3 x_4 x_5 x_7 x_9 x_{11} x_{14} t_0 t_2 t_3 t_4$	$10k_2 + 9$
6	22	$x_0 x_2 x_3 x_4 x_5 x_7 x_9 x_{11} x_{15} t_0 t_2 t_4$	$97k_3 + 89$
6	22	$x_0 x_3 x_4 x_5 x_7 x_9 x_{11} x_{14} t_0 t_1 t_3 t_4$	$111k_1 + 124$

Target Instance:

- $\text{Enc}_k : \mathbb{F}_{127}^{16} \mapsto \mathbb{F}_{127}^{16}$
- 16 plaintext (x_i), 16 tweak (t_i) variables

Compute Environment:

- **GPU:** NVIDIA A40 (46GB VRAM)
- **Framework:** CUDA-based Kite-Attack

Thank You for Your Attention!